

Tema 3 — Anexo 0: dónde vive el agente

Slides independientes del deck principal de Tema 3 — se dan primero, antes del Anexo A (Agent Identity) y el Anexo B (OPA/Rego)

Antes de SPIFFE y Rego: dónde vive el agente

- ¿Dónde corre, en términos de arquitectura, un agente de IA en la nube? — determina qué invariantes de ejecución garantiza la plataforma y cuáles quedan bajo tu responsabilidad
- Producto: **Agent Engine** — doc también dice **Agent Runtime** — recurso `reasoningEngines` de la API de Vertex AI
- Dentro de **Gemini Enterprise Agent Platform** (rebrand de Vertex AI, Cloud Next abril 2026)

Cinco ejes de comparación

VM vs. Docker/Cloud Run vs. Agent Runtime, comparados en:

1. Unidad de abstracción
2. Control de infraestructura
3. Primitivas del runtime
4. Modelo de persistencia de estado
5. Modelo de deployment

No hay "mejor" en abstracto — hay más adecuado según el sistema que se construya

Unidad de abstracción y control de infraestructura

	Compute Engine	Cloud Run	Agent Runtime
Unidad	Hardware virtualizado	Proceso aislado	Entidad agéntica (código de app)
Control infra	Total (networking, sysctl, parches OS)	Medio (imagen, puertos, auto-scaling)	Nulo — serverless estricto

- Agent Runtime: no hay imagen que administrar en sentido clásico — aunque por debajo sigue existiendo un contenedor

Primitivas del runtime y persistencia de estado

- VM: CPU, RAM, IP, disco. Cloud Run: requests, concurrencia, auto-scaling
- Agent Runtime: sesiones, contexto/memoria semántica, ejecución de herramientas, guardrails
- Cloud Run es **stateless** por diseño → cablear DB externa (Memorystore/Firestore)
- Agent Runtime: el mismo código de framework se conecta, sin cambios de lógica, a un servicio de persistencia gestionado

El bucle ReAct es código: el Runner y el Event Loop de ADK

- **Corrección importante:** "Agent Runtime entiende la semántica y maneja el bucle" ≠ intérprete especial de la plataforma
- El bucle lo implementa el **framework** — en ADK, el `Runner` : *"orchestrator para una invocación de usuario único"* (doc oficial)
- Mecanismo: generadores Python con `yield` — la lógica del agente cede un `Event` , se **pausa inmediatamente**, y solo se reanuda tras que el `Runner` lo procese y comprometa vía `SessionService`

Estructura de un Event (ADK)

- `content` — texto de respuesta o llamada de función
- `author` — quién generó el evento
- `actions` (`EventActions`): `state_delta` (cambios de estado) + `artifact_delta` (cambios de artefactos)
- `partial` — indicador de streaming
- Una llamada a herramienta = un `Event` con `content` = llamada de función, procesado por el mismo mecanismo yield/reanudación

SessionService y MemoryService: de local a gestionado

- El `Runner` necesita, como dependencia inyectable: `SessionService` (clase base `BaseSessionService`, ref. local `InMemorySessionService`) y opcionalmente `MemoryService` (`BaseMemoryService`)
- Local: implementación en memoria. En Agent Runtime: mismo código, conectado a **Sessions API** y **Memory Bank** gestionados
- No es que la plataforma "entienda" conversaciones — te da el backend que el framework ya pedía como pieza enchufable

Diferencia 1 — Orientación al dominio agéntico

- VM/Cloud Run: solo señales de red (TCP/HTTP) y ciclo de vida de proceso (SIGTERM, CPU, memoria)
- Agent Runtime: contrato de hosting para agentes — aloja el objeto serializado, expone `reasoningEngines:query`, sin boilerplate HTTP
- Quien "entiende" el bucle sigue siendo el framework (el `Runner`) — la plataforma evita que lo envuelvas vos en infraestructura de servicio

Diferencia 2 — Gestión de estado y memoria integrada

- Stateless en Cloud Run = cablear a mano historial, contexto, embeddings de memoria (Firestore + vector DB)
- Agent Runtime: `SessionService` / `MemoryService` conectados a Sessions API / Memory Bank automáticamente
- Pausar, reanudar, inspeccionar una sesión sin código de persistencia adicional

Diferencia 3 — Sandboxing de herramientas

- Un contenedor aísla el proceso principal del host, pero NO aísla nada dentro del proceso
- Código dinámico (Python de análisis) o tools de terceros con input no confiable = superficie de ataque real (mismo espacio de memoria y credenciales)
- Agent Engine integra **code execution sandboxes** aislados, separados del proceso que aloja al Runner

Diferencia 4 — Observabilidad y gobernanza específica

- Logging/Monitoring tradicional: señales genéricas de infra (CPU, latencia HTTP, p99)
- Cada paso del bucle ya es un `Event` estructurado (`content` + `actions` tipados) → trazabilidad del árbol de pensamiento sin instrumentación extra
- Integración nativa con **Model Armor** / **Agent Gateway** — tema de los Anexos A y B

Cómo se relaciona por debajo: contenedores y serialización, no magia

- Agent Engine se apoya en contenedores, no los reemplaza — el bucle sigue siendo código del framework
- (*fuentes: análisis técnico externo, no detallado así en doc oficial*) — con ADK: el objeto vivo del agente se serializa con **cloudpickle**, sube a un bucket, se "descongela" en un servidor de Vertex AI
- Otros frameworks (LangChain, LlamaIndex): contenedor sobre infra serverless, API ReasoningEngine
- Soporta **BYOC**: imagen propia si cumple el Runtime Contract

Transición

- Terreno técnico común resuelto: qué tipo de infraestructura es Agent Engine, y qué hace el framework vs. la plataforma
- **Anexo A:** cómo prueba su identidad un agente acá — SPIFFE vía Agent Identity
- **Anexo B:** quién decide qué puede hacer — PEP, PDP, y el lugar de Rego

Fuentes

- [Event Loop | Agent Development Kit \(ADK\)](#)
- [Use an Agent Development Kit agent | Vertex AI Agent Builder | Google Cloud Documentation](#)
- [Quickstart with Agent Development Kit | Vertex AI Agent Builder | Google Cloud Documentation](#)